

Setup Instructions:

To run this messaging backend, make sure you have Node.js installed. Then follow these steps:

- Clone the repository and navigate to the project directory in your terminal.
- Run `npm install` to install all the dependencies.
- Set up a `.env` file in the root folder and include your Firebase admin credentials and any other necessary environment variables.
- Make sure you have a valid `serviceAccountKey.json` file from your Firebase project and place it in the root or where your `firebase.js` file expects it.
- Start the server by running `node index.js`. You should see "Server is running on port 5050" in the console if everything is working correctly.
- You can now use tools like Postman to interact with the endpoints for sending messages, creating groups, editing profile info, and more.

Structure and Feature Decisions:

For simplicity and clarity, the backend is organized into separate route files (**auth.js** and **messages.js**). All Firebase logic is centralized using the Firebase Admin SDK via a **firebase.js** file.

We decided to support both private and group messaging. Private messages include the sender, receiver, message content, timestamp, and type. Group messages are stored with a group ID instead of a receiver, and there's logic for group creation, adding/removing members, and sending messages.

To keep data queries efficient and predictable, we used a `chatId` formed by combining and sorting the two users involved in private chats. This allows us to fetch messages between two users in a consistent and optimized way.

We also included support for profile updates and account deletion. For testing purposes, Postman collections were created with sample requests for all endpoints.